A Mini Project Report on

String Matching Algorithms: Naive vs Rabin-Karp – A Comparative Analysis

Submitted By

Rohit Sharma 7445

Subject: Design and Analysis of Algorithms

Class: B.E. Computer B

Submitted To: Prof. Rushali Katkar

Submitted as a partial fulfillment towards Term I of

B.E. Computer Engineering



ARMY INSTITUTE OF TECHNOLOGY

Department of Computer Engineering



Savitribai Phule Pune University

2026-27

ARMY INSTITUTE OF TECHNOLOGY

Department of Computer Engineering



C E R T I F I C A T E

This is to certify that **Rohit Sharma (Roll No. 7445)** of Army Institute of Technology, Dighi, Pune has successfully completed the mini project titled "**String Matching Algorithms: Naive vs Rabin-Karp – A Comparative Analysis**" in the subject **Design and Analysis of Algorithms** and submitted this report as a partial fulfillment of Term – I for award of **Final Year** of Bachelor of Computer Engineering (Class: B.E. Computer B), from Savitribai Phule Pune University, Pune, during the Academic Year **2026-27**.

Prof. Rushali Katkar	Dr. S. R. Dhore
Subject Teacher	**H.O.D.**

ACKNOWLEDGMENT

I would like to express my sincere gratitude to my subject teacher and project guide, **Prof. Rushali Katkar**, for her valuable guidance, constructive suggestions and constant encouragement throughout the course of this mini project. Her clear explanation of the design and analysis of algorithms helped me to understand how theoretical complexity results can be verified through experiments.

I am deeply thankful to **Dr. S. R. Dhore**, Head of the Department of Computer Engineering, for providing the academic environment and the laboratory facilities that were necessary to carry out this work successfully.

I also extend my appreciation to the authors of the textbooks and research papers on string matching, especially the work of Karp and Rabin, which formed the foundation of this study, and to my classmates and friends for their helpful discussions and feedback while testing the programs.

Finally, I express my sincere gratitude to the **Army Institute of Technology, Pune**, for its constant encouragement and for including mini projects as an integral part of the curriculum, thereby enabling me to bridge the gap between theoretical knowledge and practical application. I am also grateful to my family for their continued support.

Rohit Sharma
Roll No. 7445, B.E. Computer B

ABSTRACT

String matching, the task of finding every occurrence of a pattern inside a larger text, is one of the most fundamental problems in computer science. It lies behind text editors, search engines, compilers, plagiarism checkers, intrusion-detection systems and DNA sequence analysis. This mini project, carried out for the subject **Design and Analysis of Algorithms**, implements and compares two classical solutions in C++: the **Naive (brute-force) algorithm** and the **Rabin-Karp algorithm**, which uses a rolling hash to avoid most character comparisons.

Both algorithms were implemented as functions that return the starting index of every match, including overlapping matches. A test harness runs them on fourteen carefully chosen test cases, which include ordinary text, patterns that do not occur, patterns longer than the text, single-character patterns, overlapping matches, case-sensitivity checks and four large worst-case style inputs of up to 200,000 characters. Each algorithm is executed 100 times per test and the average running time is reported in microseconds.

The results show that both algorithms always report identical match positions, which confirms correctness. On very small inputs both complete in well under a microsecond and neither is consistently faster, because the hashing overhead of Rabin-Karp cancels its savings. On large inputs with long, repetitive patterns, however, Rabin-Karp was between **103 and 876 times faster**, since the Naive algorithm needed up to about 100 million character comparisons while Rabin-Karp needed none or only one verification. The report also explains the rolling hash, spurious hits, the best, average and worst case complexity of each method, and the practical limitations of the chosen hash parameters.

Key outcomes of this project include:

- Working C++ implementations of the Naive and Rabin-Karp string matching algorithms.
- A repeatable benchmark that confirms the O(n) behaviour of Rabin-Karp and the O(nm) worst case of the Naive method.
- A clear comparison of when each algorithm is preferable.

TABLE OF CONTENTS

Chapter 1: Introduction 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope and Limitations 1
Chapter 2: Theoretical Background 2
2.1 Basic Terminology 2
2.2 Classification of String Matching Algorithms 2
2.3 Hashing Basics 2
Chapter 3: The Naive String Matching Algorithm 3
3.1 Idea 3
3.2 Flowchart 3
3.3 Algorithm 3
3.4 Worked Example 4
3.5 Complexity Analysis 4
Chapter 4: The Rabin-Karp Algorithm 5
4.1 Idea 5
4.2 The Rolling Hash 5
4.3 Algorithm 5
4.4 Flowchart 6
4.5 Worked Example 6
4.6 Spurious Hits 7
4.7 Complexity Analysis 7
4.8 Choice of the Parameters d and q 7
Chapter 5: Implementation 8
5.1 Development Environment 8
5.2 Program Structure 8
5.3 Source Code 8
5.4 Explanation of the Code 10
5.5 Timing Methodology 11
Chapter 6: Test Cases and Results 12
6.1 Design of the Test Cases 12
6.2 Program Output 12
6.3 Verification of Correctness 14
Chapter 7: Performance Analysis 15
7.1 Small Inputs (T1–T10) 15
7.2 Large Inputs (T11–T14) 16
7.3 Operation Counts 17
7.4 Cost per Operation 17
7.5 Observations 18
Chapter 8: Discussion 19
8.1 Comparison of the Two Algorithms 19
8.2 Applications 19
8.3 When Should Each Algorithm Be Used? 19
Chapter 9: Conclusion and Future Scope 20
9.1 Conclusion 20
9.2 Future Scope 20
References 20

LIST OF FIGURES

Figure 3.1: Flowchart of the Naive algorithm..... 3
Figure 4.1: Flowchart of the Rabin-Karp algorithm 6
Figure 7.1: Running time on small inputs (T1–T10) 15
Figure 7.2: Running time on large inputs (logarithmic vertical axis)..... 16
Figure 7.3: Speed-up of Rabin-Karp over the Naive algorithm..... 16

LIST OF TABLES

Table 2.1: Common string matching algorithms 2
Table 3.1: Naive algorithm on text "AABAACAADAABAABA", pattern "AABA" 4
Table 4.1: Window hash values for text "ABABDABACDABABCABAB", pattern hash = 88.... 6
Table 5.1: Functions of the program..... 8
Table 6.1: Test cases (n = text length, m = pattern length)..... 12
Table 7.1: Average running time on small inputs (microseconds) 15
Table 7.2: Average running time on large inputs (microseconds)..... 16
Table 7.3: Character comparisons and hash hits (single run) 17
Table 7.4: Time per basic operation on large inputs..... 17
Table 8.1: Naive versus Rabin-Karp..... 19

Chapter 1: Introduction

1.1 Background

Searching for a word in a document is something every computer user does many times a day, yet behind that simple action lies a classical algorithmic problem known as **string matching** or pattern matching. Given a text of length n and a pattern of length m, the task is to find all positions in the text at which the pattern occurs. The efficiency of the solution matters a great deal when the text is very large, such as a genome or the network traffic that an intrusion-detection system must inspect.

In the subject Design and Analysis of Algorithms we study how to design algorithms, prove them correct and measure their cost in terms of time and space. String matching is an excellent topic for this study because it contains simple brute-force solutions, clever hashing-based solutions such as Rabin-Karp, automaton-based solutions such as Knuth-Morris-Pratt, and heuristics such as Boyer-Moore. This mini project concentrates on the first two, the **Naive algorithm** and the **Rabin-Karp algorithm**, and compares them both theoretically and experimentally.

1.2 Problem Statement

To design, implement and compare the Naive and Rabin-Karp string matching algorithms in C++, so that for any given text and pattern both algorithms report all (including overlapping) occurrences of the pattern, and to analyse their running time on small inputs and on large inputs that stress the worst case, in order to determine the situations in which each algorithm is preferable.

1.3 Objectives

- To understand the working of the Naive and the Rabin-Karp string matching algorithms.
- To derive the rolling hash function used by Rabin-Karp and to understand the idea of spurious hits.
- To state and justify the best, average and worst case time complexity of both algorithms.
- To implement both algorithms in C++ so that they return all match positions.
- To design a set of test cases that checks correctness (matches, no match, overlapping matches, pattern longer than text, case sensitivity) and performance (large worst-case inputs).
- To measure and compare the running time of both algorithms and to explain the observed results using operation counts.

1.4 Scope and Limitations

The project is restricted to exact, single-pattern string matching over 8-bit characters. Approximate matching, regular expressions and Unicode-aware searching are outside the scope of this work. Running times were measured on one personal computer using the standard `std::chrono` library; absolute values will differ on other machines, but the relative behaviour of the two algorithms is the quantity of interest.

Chapter 2: Theoretical Background

2.1 Basic Terminology

An **alphabet** Σ is a finite set of symbols, for instance the 256 possible values of an 8-bit character. A **string** is a finite sequence of symbols drawn from Σ. The **text** T[0..n−1] is the string that is searched, and the **pattern** P[0..m−1] is the string being looked for, with m ≤ n for a match to be possible. A pattern **occurs with shift s** in the text if T[s..s+m−1] = P[0..m−1]. The problem is to find all valid shifts s in the range 0 ≤ s ≤ n−m.

A **window** is the substring of the text of length m currently aligned with the pattern. Occurrences may **overlap**: the pattern `aaa` occurs three times in `aaaaa`, at shifts 0, 1 and 2, and a correct algorithm must report all of them.

2.2 Classification of String Matching Algorithms

Exact string matching algorithms are commonly grouped by the technique that they use. Table 2.1 summarises the best-known algorithms; the first two are the subject of this project. The Naive method is the natural baseline, and Rabin-Karp keeps its sliding-window structure but replaces most character comparisons by one integer comparison, so comparing them isolates the effect of hashing.

Table 2.1: Common string matching algorithms

Algorithm	Technique	Preprocessing	Matching time (worst)
Naive (brute force)	Compare at every shift	None	O(nm)
Rabin-Karp	Rolling hash + verification	O(m)	O(nm); expected O(n + m)
Knuth-Morris-Pratt	Prefix function (failure table)	O(m)	O(n)
Boyer-Moore	Bad-character and good-suffix shifts	O(m + |Σ|)	O(nm); fast in practice
Finite automaton	DFA built from the pattern	O(m|Σ|)	O(n)

2.3 Hashing Basics

A **hash function** maps a string to a small integer called its hash value. Two equal strings must always produce equal hash values, but two different strings may sometimes produce the same value; this event is called a **collision**. Rabin-Karp uses a **polynomial hash**. Treating the characters of a string s of length m as digits in base d (here d = 256, the number of possible characters), the hash is

$$H(s) = (s[0]\cdot d^{(m-1)} + s[1]\cdot d^{(m-2)} + \ldots + s[m-1]) \bmod q$$

where q is a prime modulus. The modulus keeps the value small enough to fit in a machine word and avoids integer overflow. The key property that makes the method efficient is that the

hash of the next window can be computed from the hash of the current window in constant time, as shown in Chapter 4.

Chapter 3: The Naive String Matching Algorithm

3.1 Idea

The Naive algorithm, also called the brute-force algorithm, slides the pattern over the text one position at a time. At every shift i from 0 to n−m it compares the pattern with the window T[i..i+m−1] character by character, starting from the left. The comparison stops as soon as a mismatching pair is found or when all m characters have matched. If all m characters match, the shift i is recorded as a valid occurrence. After the comparison, whether it succeeded or failed, the pattern is moved one position to the right. Overlapping matches are therefore found automatically.

3.2 Flowchart

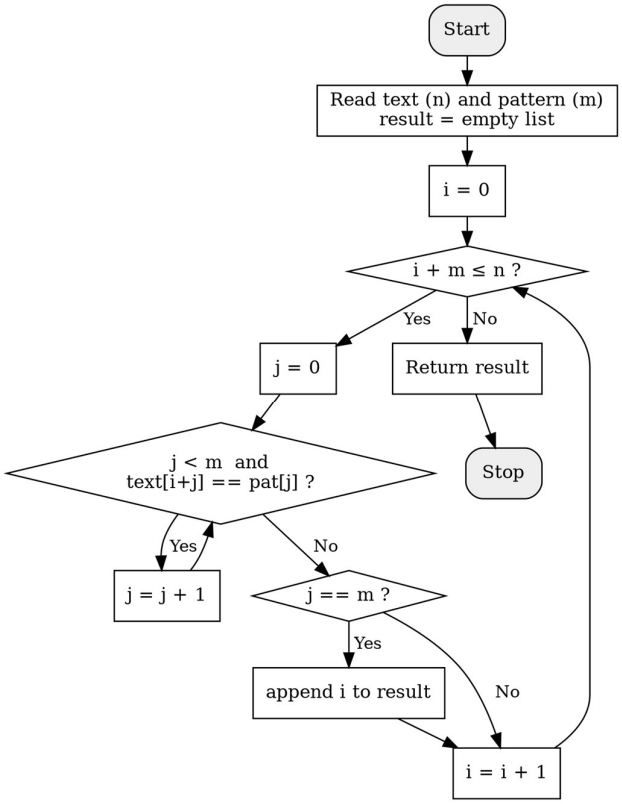


Figure 3.1: Flowchart of the Naive algorithm

3.3 Algorithm

```
NAIVE-MATCH(text, pat)
1  n = length(text);  m = length(pat)
2  result = empty list
3  for i = 0 to n - m
4       j = 0
5       while j < m and text[i + j] == pat[j]
6           j = j + 1
```

```
7       if j == m
8           append i to result
9   return result
```

Algorithm 3.1: Naive string matching (pseudocode)

3.4 Worked Example

Consider the text `AABAACAADAABAABA` (n = 16) and the pattern `AABA` (m = 4), which is the second test case of this project. There are n − m + 1 = 13 possible shifts. Table 3.1 shows, for every shift, the window, the number of character comparisons made (counting the final mismatching comparison) and the outcome.

Table 3.1: Naive algorithm on text "AABAACAADAABAABA", pattern "AABA"

Shift i	Window	Comparisons	Outcome
0	AABA	4	Match at 0
1	ABAA	2	Mismatch at pattern position 1
2	BAAC	1	Mismatch at pattern position 0
3	AACA	3	Mismatch at pattern position 2
4	ACAA	2	Mismatch at pattern position 1
5	CAAD	1	Mismatch at pattern position 0
6	AADA	3	Mismatch at pattern position 2
7	ADAA	2	Mismatch at pattern position 1
8	DAAB	1	Mismatch at pattern position 0
9	AABA	4	Match at 9
10	ABAA	2	Mismatch at pattern position 1
11	BAAB	1	Mismatch at pattern position 0
12	AABA	4	Match at 12

The pattern is found at shifts 0, 9 and 12, which agrees with the program output in Chapter 6. The total number of comparisons is 30, compared with the 52 that would be needed if every window were compared in full.

3.5 Complexity Analysis

Best case. The best case occurs when the first character of the pattern does not appear in the text at all. Each shift then costs exactly one comparison, so the total number of comparisons is n − m + 1 and the running time is O(n).

Worst case. The worst case occurs when almost every window matches the pattern for nearly all m characters before failing, for example the text a^n and the pattern a^(m−1)b. Each of the n − m + 1 shifts then performs about m comparisons, giving a running time of Θ((n − m + 1)·m), which is O(nm) when m is not close to n. Test cases 11 to 14 are of this kind.

Average case. For random text over an alphabet of size k, the probability that the first j characters of the window match the pattern is 1/k^j, so the expected number of comparisons per shift is $1 + 1/k + 1/k^2 + \ldots \leq k/(k-1)$, which is at most 2 for $k \geq 2$. The expected total is therefore O(n), so the Naive algorithm is acceptable on ordinary text.

Space. Apart from the output list, the algorithm needs only a constant number of integer variables, so its auxiliary space is O(1).

Chapter 4: The Rabin-Karp Algorithm

4.1 Idea

The Rabin-Karp algorithm was proposed by Michael O. Rabin and Richard M. Karp in 1987. Like the Naive method it examines every window of length m, but it first compares a **hash value** of the pattern with the hash value of the window. If the two hash values differ, the strings are certainly different and the window is rejected at the cost of one integer comparison. Only when the hash values are equal does the algorithm compare the characters, to make sure that the match is real and not a collision. The cleverness of the method lies in computing the hash of each new window from the previous one in O(1) time, instead of recomputing it from scratch in O(m) time.

4.2 The Rolling Hash

Let the current window be T[i..i+m−1] with hash value t_i = (T[i]·d^(m−1) + T[i+1]·d^(m−2) + … + T[i+m−1]) mod q. The next window T[i+1..i+m] has hash value t_(i+1). To obtain t_(i+1) from t_i we remove the contribution of the outgoing leftmost character T[i], shift all remaining digits one place to the left by multiplying by d, and add the incoming character T[i+m]:

$$t_{(i+1)} = (d \cdot (t_i - T[i] \cdot h) + T[i+m]) \bmod q, \quad \text{where } h = d^{(m-1)} \bmod q$$

The constant h is computed once in a loop of m − 1 multiplications. In the program this is line 49: `t = (d * (t - text[i] * h) + text[i + m]) % q;`. Since the C++ remainder operator can return a negative result when its left operand is negative, the program adds q whenever t becomes negative so that the hash stays in the range 0 to q − 1.

4.3 Algorithm

```
RABIN-KARP(text, pat, d, q)
1   n = length(text);  m = length(pat)
2   if m > n then return empty list
3   h = d^(m-1) mod q
4   p = 0;  t = 0
5   for i = 0 to m - 1
6       p = (d * p + pat[i]) mod q
7       t = (d * t + text[i]) mod q
8   for i = 0 to n - m
9       if p == t
10          if text[i..i+m-1] == pat[0..m-1]   // verify
11              append i to result
12      if i + m < n
13          t = (d * (t - text[i] * h) + text[i+m]) mod q
14          if t < 0 then t = t + q
15  return result
```

Algorithm 4.1: Rabin-Karp string matching (pseudocode)

4.4 Flowchart

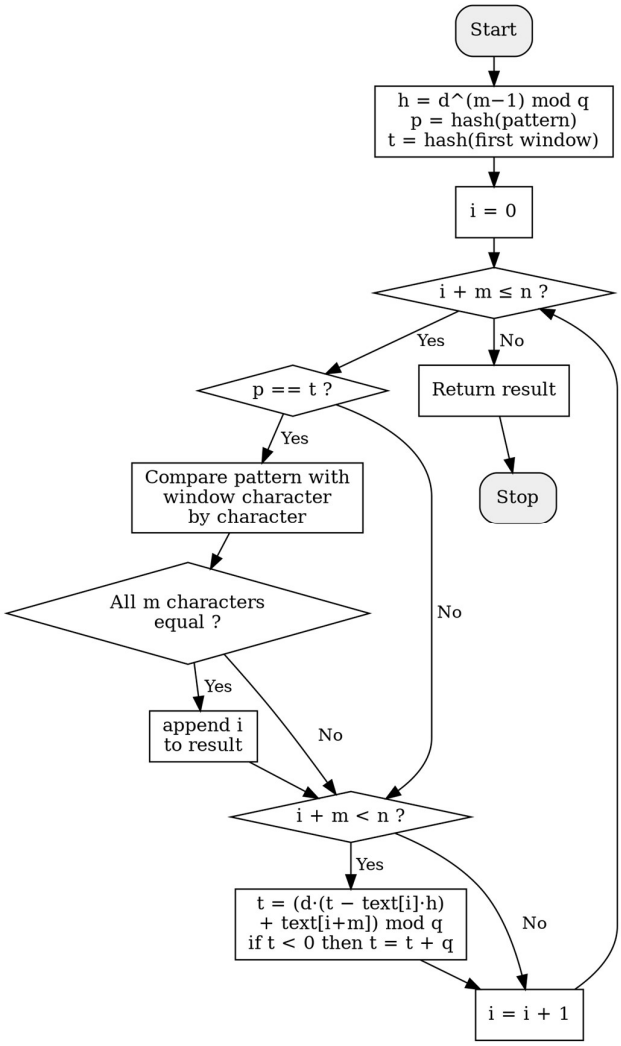


Figure 4.1: Flowchart of the Rabin-Karp algorithm

4.5 Worked Example

Take the text `ABABDABACDABABCABAB` (n = 19) and the pattern `ABABCABAB` (m = 9), the first test case, with d = 256 and q = 101. The hash value of the pattern is H(P) = 88. Table 4.1 lists the hash value of each of the 11 windows.

Table 4.1: Window hash values for text "ABABDABACDABABCABAB", pattern hash = 88

Shift i	Window	Hash	Action
0	ABABDABAC	56	hash differs, skip
1	BABDABACD	17	hash differs, skip
2	ABDABACDA	5	hash differs, skip
3	BDABACDAB	89	hash differs, skip
4	DABACDABA	55	hash differs, skip

Shift i	Window	Hash	Action
5	ABACDABAB	91	hash differs, skip
6	BACDABABC	88	hash equal, verified: spurious hit
7	ACDABABCA	1	hash differs, skip
8	CDABABCAB	75	hash differs, skip
9	DABABCABA	83	hash differs, skip
10	ABABCABAB	88	hash equal, verified: MATCH

Nine of the eleven windows are rejected immediately by one integer comparison. At shift 6 the window `BACDABABC` happens to have the same hash value 88 as the pattern although the strings are different; this is a **spurious hit** (collision). The character-by-character verification detects it at once because the first characters `B` and `A` differ. At shift 10 the hash values are equal and verification confirms the true match. This agrees with the operation counts later measured by an instrumented run: the Naive algorithm made 29 comparisons on this test, while Rabin-Karp made only 10 character comparisons, all of them during verification.

4.6 Spurious Hits

Because the number of possible strings of length m is far greater than the number of hash values q, collisions cannot be avoided. A spurious hit costs up to m extra comparisons for verification. If q is a reasonably large prime, the probability that two unequal windows collide is about 1/q, so the expected number of spurious hits over n windows is about n/q, and the expected running time is O(n + m) as long as the number of real matches is small. With the small prime q = 101 used here the collision probability is about 1 percent per window, acceptable for a demonstration but too high for production use.

4.7 Complexity Analysis

Preprocessing. Computing h, p and the first window hash t takes O(m) time. **Matching.** The loop runs n − m + 1 times; each iteration does an O(1) hash comparison and an O(1) hash update, so the cost without verification is O(n). **Verification.** Each real match costs m comparisons, and each spurious hit costs at most m comparisons.

- **Best and average case: O(n + m)** when there are few matches and few collisions.
- **Worst case: O(nm)**, which occurs if every window has the same hash as the pattern, for example when the pattern occurs at almost every shift (text `aaaa...a`, pattern `aaa...a`) or when collisions are frequent. In that situation verification is done at nearly every shift, just like in the Naive algorithm.
- **Auxiliary space: O(1)**, only a handful of integer variables.

4.8 Choice of the Parameters d and q

The base d should be at least the size of the alphabet; d = 256 is the natural choice for 8-bit characters. The modulus q should be a prime so that the hash values are spread evenly. A larger prime reduces collisions but the intermediate value d · (t − T[i]·h) must still fit in the integer type. With d = 256 and q = 101 the largest intermediate value is below 6.6 million, which is safe for a 32-bit `int`; a prime near 10^9 would reduce collisions dramatically but would need 64-bit arithmetic.

Chapter 5: Implementation

5.1 Development Environment

Both algorithms were implemented in a single C++ source file named `mini_project.cpp`. The program was written in Visual Studio Code on a Windows computer and compiled and run from the PowerShell terminal with the commands `g++ mini_project.cpp -o mini_project` followed by `.\mini_project`. The program uses only the standard library headers `<iostream>`, `<string>`, `<vector>` and `<chrono>`; no external library is required. It is important to note that the program was compiled without any optimisation flag, so the absolute timings reported in Chapters 6 and 7 are larger than those that would be obtained with `-O2`.

5.2 Program Structure

The program is organised into five functions, listed in Table 5.1. The two matching functions take the text and the pattern as constant references, which avoids copying large strings, and return a `vector<int>` that contains the starting index of every match. An empty vector means that the pattern was not found.

Table 5.1: Functions of the program

Function	Lines	Purpose
naive(text, pat)	7–20	Brute-force search; returns all match positions
rabinKarp(text, pat)	21–55	Rolling-hash search with d = 256 and q = 101; returns all match positions
print(v)	56–63	Prints the positions or the message "not found"
test(text, pat)	64–84	Runs both algorithms 100 times each, measures the average time and prints the results
main()	85–107	Defines the 14 test cases, including generated large inputs

5.3 Source Code

The complete source code of the project is given below (black text on a white background, with line numbers added for reference).

```
 1  #include <iostream>
 2  #include <string>
 3  #include <vector>
 4  #include <chrono>
 5  using namespace std;
 6  using namespace chrono;
 7  vector<int> naive(const string &text, const string &pat)
 8  {
 9      vector<int> result;
10      int n = text.size(), m = pat.size();
11      for (int i = 0; i + m <= n; i++)
12      {
13          int j = 0;
```

```
 14          while (j < m && text[i + j] == pat[j])
 15              j++;
 16          if (j == m)
 17              result.push_back(i);
 18      }
 19      return result;
 20  }
 21  vector<int> rabinKarp(const string &text, const string &pat)
 22  {
 23      vector<int> result;
 24      int n = text.size(), m = pat.size();
 25      if (m > n)
 26          return result;
 27      int d = 256, q = 101;
 28      int h = 1;
 29      for (int i = 0; i < m - 1; i++)
 30          h = (h * d) % q;
 31      int p = 0, t = 0;
 32      for (int i = 0; i < m; i++)
 33      {
 34          p = (d * p + pat[i]) % q;
 35          t = (d * t + text[i]) % q;
 36      }
 37      for (int i = 0; i + m <= n; i++)
 38      {
 39          if (p == t)
 40          {
 41              int j = 0;
 42              while (j < m && text[i + j] == pat[j])
 43                  j++;
 44              if (j == m)
 45                  result.push_back(i);
 46          }
 47          if (i + m < n)
 48          {
 49              t = (d * (t - text[i] * h) + text[i + m]) % q;
 50              if (t < 0)
 51                  t += q;
 52          }
 53      }
 54      return result;
 55  }
 56  void print(const vector<int> &v)
 57  {
 58      if (v.empty())
 59          cout << "not found";
 60      for (int x : v)
 61          cout << x << " ";
 62      cout << endl;
 63  }
 64  void test(string text, string pat)
 65  {
 66      cout << "Text    : " << (text.size() < 50 ? text : "(large text)") << endl;
 67      cout << "Pattern : " << (pat.size() < 50 ? pat : "(large pattern)") << endl;
 68      int runs = 100, count = 0;
 69      auto start = high_resolution_clock::now();
 70      for (int i = 0; i < runs; i++)
 71          count += naive(text, pat).size();
 72      double naiveTime = duration<double, micro>(high_resolution_clock::now() -
start).count() / runs;
 73      start = high_resolution_clock::now();
 74      for (int i = 0; i < runs; i++)
 75          count += rabinKarp(text, pat).size();
 76      double rkTime = duration<double, micro>(high_resolution_clock::now() -
start).count() / runs;
 77      cout << "Naive      found at: ";
 78      print(naive(text, pat));
 79      cout << "Rabin-Karp found at: ";
 80      print(rabinKarp(text, pat));
 81      cout << "Naive time      : " << naiveTime << " us" << endl;
 82      cout << "Rabin-Karp time : " << rkTime << " us" << endl;
 83      cout << "----------------------------------------" << endl;
 84  }
 85  int main()
 86  {
 87      test("ABABDABACDABABCABAB", "ABABCABAB");
 88      test("AABAACAADAABAABA", "AABA");
```

```
 89      test("HELLO WORLD", "XYZ");
 90      test("abcdefg", "abc");
 91      test("abcdefg", "efg");
 92      test("hello", "hello");
 93      test("hi", "hello");
 94      test("banana", "a");
 95      test("aaaaa", "aaa");
 96      test("Hello hello HELLO", "hello");
 97      test(string(100000, 'a'), string(100, 'a') + "b");
 98      test(string(200000, 'a'), string(500, 'a') + "b");
 99      test(string(50000, 'a') + "b", string(1000, 'a') + "b");
100      string t, p;
101      for (int i = 0; i < 100000; i++)
102          t += "ab";
103      for (int i = 0; i < 300; i++)
104          p += "ab";
105      test(t, p + "c");
106      return 0;
107  }
```

Listing 5.1: mini_project.cpp

5.4 Explanation of the Code

The naive function

For every shift i such that the pattern still fits inside the text (`i + m <= n`), the inner `while` loop advances j as long as the characters agree. If the loop ends with j equal to m, every character matched and i is appended to the result. The check `i + m <= n` also takes care of the case where the pattern is longer than the text: the loop body never executes and an empty vector is returned.

The rabinKarp function

The function first returns an empty result when m > n. It then fixes d = 256 and q = 101 and computes h = d^(m−1) mod q with a loop of m − 1 steps. The next loop (lines 34–35) computes the pattern hash p and the hash t of the first window together, using the Horner form p = (d·p + pat[i]) mod q, which needs only one multiplication and one addition per character. The main loop compares p with t; only if they are equal does it run the verification loop, which is the same character comparison as in the Naive algorithm. The statement `if (i + m < n)` prevents reading beyond the end of the text when the last window has been processed, and the rolling update on line 49 prepares the hash of the next window. A final correction adds q if the remainder is negative.

The test function

The function prints the text and the pattern, but replaces any string of 50 characters or more by the words "(large text)" and "(large pattern)" so that the output remains readable. It then calls each algorithm `runs = 100` times and divides the elapsed time by 100 to obtain the average time per call. The running totals added to the variable `count` make sure that the compiler cannot discard the calls as unused. Finally, the results are printed once more so that the positions found by the two algorithms can be compared visually.

Design decisions and limitations

- Both functions are case sensitive, because they compare raw character codes; the pattern `hello` is therefore not found in `HELLO`.
- Overlapping occurrences are reported because the pattern always moves ahead by exactly one position.
- Sizes are stored in `int`, which is adequate for the inputs used (up to 200,000 characters) but would need `size_t` or `long long` for texts larger than 2 GB.
- An empty pattern (m = 0) is not handled specially; the functions would report a match at every position. This case does not occur in the test set.

5.5 Timing Methodology

Each timing uses `std::chrono::high_resolution_clock`. The clock is read before and after a loop of 100 calls, and the difference is converted to microseconds with `duration<double, micro>` and divided by 100. The measured time therefore includes the cost of creating and destroying the result vector but not the cost of printing. Averaging over 100 runs reduces random variation, but it cannot remove it completely. For inputs that finish in less than a microsecond, the resolution of the clock, cache effects and background activity of the operating system are of the same order as the quantity being measured, so very small differences should not be treated as significant. For the large inputs, where a single call takes hundreds of microseconds to nearly a second, the measurements are far more reliable.

Chapter 6: Test Cases and Results

6.1 Design of the Test Cases

Tests 1 to 10 are short strings that check correctness, including boundary situations. Tests 11 to 14 are large generated inputs designed to expose the O(nm) worst case of the Naive algorithm. In the table, `a^k` denotes k repeated letters a and `(ab)^k` denotes the pair ab repeated k times.

Table 6.1: Test cases (n = text length, m = pattern length)

Test	Text	Pattern	n	m	Purpose
T1	ABABDABACDABABCABAB	ABABCABAB	19	9	Textbook example; one spurious hit
T2	AABAACAADAABAABA	AABA	16	4	Several matches
T3	HELLO WORLD	XYZ	11	3	No match; characters absent
T4	abcdefg	abc	7	3	Match at the start
T5	abcdefg	efg	7	3	Match at the end
T6	hello	hello	5	5	Pattern equals text
T7	hi	hello	2	5	Pattern longer than text
T8	banana	a	6	1	Single-character pattern
T9	aaaaa	aaa	5	3	Overlapping matches
T10	Hello hello HELLO	hello	17	5	Case sensitivity
T11	a^100000	a^100 b	100,000	101	Worst case for Naive; no match
T12	a^200000	a^500 b	200,000	501	Worst case, larger input
T13	a^50000 b	a^1000 b	50,001	1,001	Single match at the very end
T14	(ab)^100000	(ab)^300 c	200,000	601	Periodic text; last char differs

6.2 Program Output

The complete program output is reproduced below as black text on a white background so that it prints economically. "found at" gives the starting indices (from 0) of the matches, and the times are averages per call in microseconds (us).

```
Text    : ABABDABACDABABCABAB
Pattern : ABABCABAB
Naive      found at: 10
Rabin-Karp found at: 10
Naive time      : 0.505 us
Rabin-Karp time : 0.435 us
----------------------------------------
Text    : AABAACAADAABAABA
Pattern : AABA
Naive      found at: 0 9 12
Rabin-Karp found at: 0 9 12
Naive time      : 0.534 us
Rabin-Karp time : 0.562 us
----------------------------------------
```

```
Text    : HELLO WORLD
Pattern : XYZ
Naive       found at: not found
Rabin-Karp found at: not found
Naive time      : 0.101 us
Rabin-Karp time : 0.114 us
----------------------------------------
Text    : abcdefg
Pattern : abc
Naive       found at: 0
Rabin-Karp found at: 0
Naive time      : 0.184 us
Rabin-Karp time : 0.145 us
----------------------------------------
Text    : abcdefg
Pattern : efg
Naive       found at: 4
Rabin-Karp found at: 4
Naive time      : 0.127 us
Rabin-Karp time : 0.145 us
----------------------------------------
```

Output 6.1: Test cases T1 to T5

```
Text    : hello
Pattern : hello
Naive       found at: 0
Rabin-Karp found at: 0
Naive time      : 0.109 us
Rabin-Karp time : 0.155 us
----------------------------------------
Text    : hi
Pattern : hello
Naive       found at: not found
Rabin-Karp found at: not found
Naive time      : 0.013 us
Rabin-Karp time : 0.012 us
----------------------------------------
Text    : banana
Pattern : a
Naive       found at: 1 3 5
Rabin-Karp found at: 1 3 5
Naive time      : 0.213 us
Rabin-Karp time : 0.225 us
----------------------------------------
Text    : aaaaa
Pattern : aaa
Naive       found at: 0 1 2
Rabin-Karp found at: 0 1 2
Naive time      : 0.247 us
Rabin-Karp time : 0.276 us
----------------------------------------
Text    : Hello hello HELLO
Pattern : hello
Naive       found at: 6
Rabin-Karp found at: 6
Naive time      : 0.269 us
Rabin-Karp time : 0.263 us
----------------------------------------
```

Output 6.2: Test cases T6 to T10

```
Text    : (large text)
Pattern : (large pattern)
Naive       found at: not found
Rabin-Karp found at: not found
Naive time      : 84064 us
Rabin-Karp time : 816.111 us
----------------------------------------
```

```
Text    : (large text)
Pattern : (large pattern)
Naive      found at: not found
Rabin-Karp found at: not found
Naive time      : 853996 us
Rabin-Karp time : 1744.92 us
---------------------------------------
Text    : (large text)
Pattern : (large pattern)
Naive      found at: 49000
Rabin-Karp found at: 49000
Naive time      : 406207 us
Rabin-Karp time : 463.519 us
---------------------------------------
Text    : (large text)
Pattern : (large pattern)
Naive      found at: not found
Rabin-Karp found at: not found
Naive time      : 512010 us
Rabin-Karp time : 1743.55 us
---------------------------------------
```

Output 6.3: Test cases T11 to T14 (large inputs)

6.3 Verification of Correctness

For all 14 tests both algorithms printed identical positions, and these agree with hand checks: T1 has its only shift at 19 − 9 = 10, T2 matches at 0, 9 and 12 (Table 3.1), T13 can only match at the end (50,001 − 1,001 = 49,000), and T10 matches only the lower-case word at 6.

Chapter 7: Performance Analysis

7.1 Small Inputs (T1–T10)

Table 7.1 compares the average times for the short inputs. The last two columns show the difference (Rabin-Karp minus Naive, in microseconds) and the faster method. Differences smaller than 0.02 microseconds were treated as a tie.

Table 7.1: Average running time on small inputs (microseconds)

Test	Naive	Rabin-Karp	Difference	Faster
T1	0.505	0.435	−0.070	Rabin-Karp
T2	0.534	0.562	+0.028	Naive
T3	0.101	0.114	+0.013	Tie (within noise)
T4	0.184	0.145	−0.039	Rabin-Karp
T5	0.127	0.145	+0.018	Tie (within noise)
T6	0.109	0.155	+0.046	Naive
T7	0.013	0.012	−0.001	Tie (within noise)
T8	0.213	0.225	+0.012	Tie (within noise)
T9	0.247	0.276	+0.029	Naive
T10	0.269	0.263	−0.006	Tie (within noise)

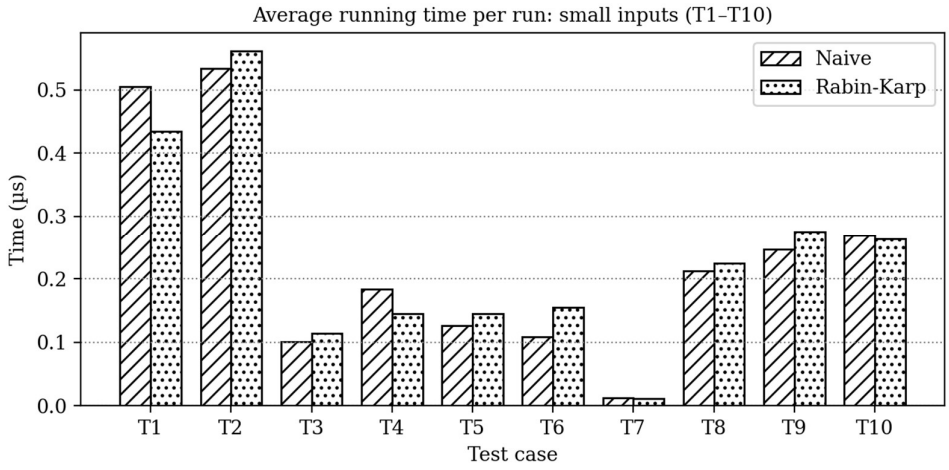


Figure 7.1: Running time on small inputs (T1–T10)

On these inputs both algorithms finish in a fraction of a microsecond (0.012 to 0.562 us) and there is no consistent winner. Using the tie threshold above, Rabin-Karp is ahead in T1 and T4 (by 0.070 and 0.039 us), the Naive algorithm is ahead in T2, T6 and T9 (by 0.028, 0.046 and 0.029 us), and the remaining five tests are ties. When the pattern is short, a mismatch is usually found at the first or second character, and Rabin-Karp must pay for the modulo arithmetic of its hash computation before any gain is possible. T6 (pattern equal to the text) and T9 (all

characters equal) show this clearly: every window must be verified anyway, so the hashing is pure overhead. Given the timer limitations described in Section 5.5, the safest conclusion is that **for short texts and short patterns the two algorithms are equivalent in practice.**

7.2 Large Inputs (T11–T14)

Table 7.2: Average running time on large inputs (microseconds)

Test	n	m	Naive (us)	Rabin-Karp (us)	Speed-up
T11	100,000	101	84,064	816.111	103.0×
T12	200,000	501	853,996	1744.92	489.4×
T13	50,001	1,001	406,207	463.519	876.4×
T14	200,000	601	512,010	1743.55	293.7×

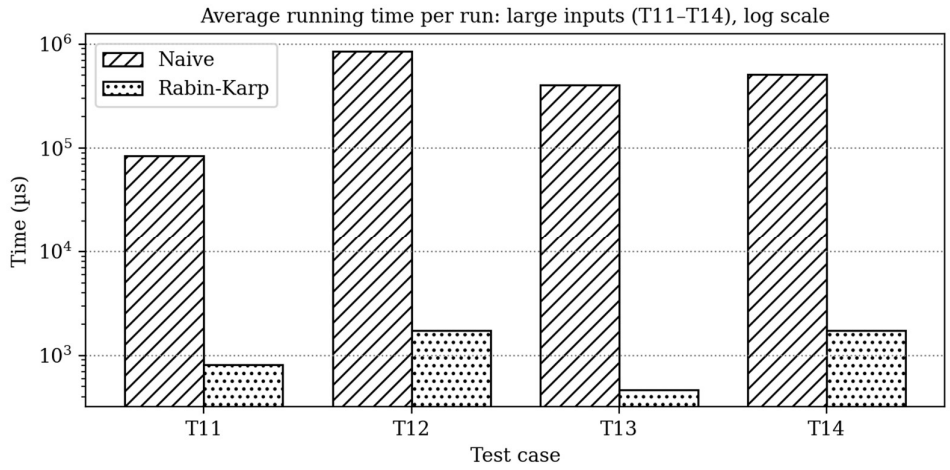


Figure 7.2: Running time on large inputs (logarithmic vertical axis)

The picture changes completely for large inputs. The Naive algorithm needed between 84 milliseconds and almost 0.85 seconds per call, whereas Rabin-Karp needed between 0.46 and 1.74 milliseconds. Because the vertical axis of Figure 7.2 is logarithmic, the gap of two to three orders of magnitude is visible even though the Rabin-Karp bars look short. Figure 7.3 shows the resulting speed-up factors, which range from about 103 times in T11 to about 876 times in T13.

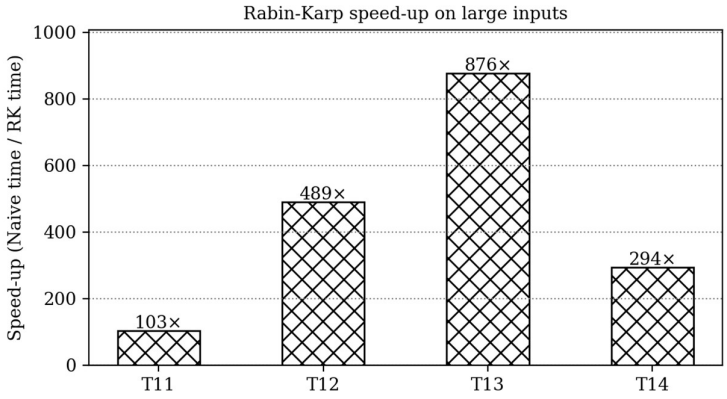


Figure 7.3: Speed-up of Rabin-Karp over the Naive algorithm

7.3 Operation Counts

To explain these timings, the two functions were run once more in an instrumented copy that counts every character comparison; for Rabin-Karp it also counts the hash hits and the spurious hits. This count does not depend on the machine, so it is an exact complement to the timing data. The results are shown in Table 7.3.

Table 7.3: Character comparisons and hash hits (single run)

Test	Naive comparisons	Rabin-Karp comparisons	RK hash hits	RK spurious hits
T1	29	10	2	1
T2	30	12	3	0
T3	9	0	0	0
T4	7	3	1	0
T5	7	3	1	0
T6	5	5	1	0
T7	0	0	0	0
T8	6	3	3	0
T9	9	9	3	0
T10	17	5	1	0
T11	10,089,900	0	0	0
T12	99,949,500	0	0	0
T13	49,050,001	1,001	1	0
T14	60,019,400	0	0	0

The counts reveal the reason for the large speed-ups. In T11, T12 and T14 the Naive algorithm performs 10.1 million, 99.9 million and 60.0 million comparisons, while Rabin-Karp performs none, because the hash value of the pattern never equals that of any window and every window is discarded by a single integer comparison. In T13 the pattern really occurs once, so Rabin-Karp performs exactly one verification of 1,001 comparisons against 49.05 million for the

Naive algorithm. Spurious hits occurred only in T1, where one occurred at shift 6 (Section 4.5); there were none in the other 13 tests.

7.4 Cost per Operation

Table 7.4 divides the measured time by the number of operations: the Naive time by its number of comparisons, and the Rabin-Karp time by its number of windows (n − m + 1), since it performs one hash comparison and one hash update per window.

Table 7.4: Time per basic operation on large inputs

Test	Naive comparisons	ns per comparison	Windows (n − m + 1)	RK ns per window
T11	10,089,900	8.33	99,900	8.17
T12	99,949,500	8.54	199,500	8.75
T13	49,050,001	8.28	49,001	9.46
T14	60,019,400	8.53	199,400	8.74

Both costs are nearly constant, about 8.3 to 8.5 ns per comparison for the Naive algorithm and 8.2 to 9.5 ns per window for Rabin-Karp. This is strong evidence that the running time of the Naive algorithm is proportional to the number of comparisons, which is of the order of n×m in these tests, whereas the running time of Rabin-Karp is proportional to n only. The latter can also be seen directly: T12 and T14 have the same text length (200,000) but different pattern lengths (501 and 601), and their Rabin-Karp times, 1744.92 and 1743.55 us, are practically identical. T11, with half the text length, took 816.111 us, about half as long. A single hash update (one multiplication and one remainder) costs about the same as a single character comparison, which is why Rabin-Karp gains only when it avoids many comparisons per window.

7.5 Observations

- Both algorithms produced identical results for all 14 tests, so the implementations are correct.
- For short inputs the difference in running time is smaller than the measurement noise; the simple Naive algorithm is entirely adequate.
- For large inputs with long, repetitive patterns the Naive algorithm exhibits its O(nm) behaviour, taking up to 0.85 s per call on 200,000 characters, whereas Rabin-Karp stays linear and takes under 2 ms.
- The running time of Rabin-Karp depends on the text length and hardly on the pattern length, as T12 and T14 show.
- The speed-up of Rabin-Karp grows with the amount of work that the Naive algorithm wastes, from about 100 times (m = 101) to about 490 times (m = 501) in T11 and T12 on all-equal text.

- The test set contains no case in which Rabin-Karp degrades to O(nm), because collisions were rare (a spurious hit occurred in only one of the 14 tests) and the pattern hash was never equal to the hash of a mismatching window in the large tests.

Chapter 8: Discussion

8.1 Comparison of the Two Algorithms

Table 8.1 summarises the strengths and weaknesses of the two algorithms, based on both the theoretical analysis and the experiments.

Table 8.1: Naive versus Rabin-Karp

Aspect	Naive	Rabin-Karp
Idea	Compare the pattern with every window character by character	Compare hash values first; compare characters only if the hashes are equal
Preprocessing	None	O(m) to compute h and the first hashes
Best / average time	O(n) typical for random text	O(n + m)
Worst-case time	O(nm) (T11–T14)	O(nm) if collisions are frequent (not observed here)
Result on large inputs	84 ms to 0.85 s	0.46 ms to 1.74 ms
Multiple patterns	Poorly suited	Well suited: compare one window hash against a set of pattern hashes
Weaknesses	Wastes information after each mismatch	Collisions; needs a good choice of d and q; arithmetic overflow risk

8.2 Applications

- **Text editors and word processors:** find and replace on documents of moderate size, where the Naive algorithm or a library routine is usually sufficient.
- **Plagiarism detection:** documents are split into fragments, whose hashes are compared; rolling hashes in the style of Rabin-Karp are the basis of many fingerprinting techniques.
- **Multiple pattern search:** searching for many keywords at once, for instance in spam filters and intrusion-detection systems, by comparing each window hash with a set of stored hash values.
- **Bioinformatics:** locating short DNA or protein sequences in very long genomes.
- **Data deduplication and file synchronisation:** rolling hashes are used to find identical blocks in different versions of a file.

8.3 When Should Each Algorithm Be Used?

The experiments suggest a simple rule. If the text is short, or the pattern is very short, the Naive algorithm should be preferred because it is simpler and equally fast. If the text is long, the pattern is long, or the text is highly repetitive, a method with linear guaranteed behaviour is required. Rabin-Karp is then a good choice, especially when several patterns are searched

simultaneously. When a guaranteed O(n + m) worst case is required, Knuth-Morris-Pratt is preferable, since the linear time of Rabin-Karp is only expected.

Chapter 9: Conclusion and Future Scope

9.1 Conclusion

In this mini project the Naive and Rabin-Karp string matching algorithms were studied, implemented in C++ and compared experimentally. The rolling hash was derived and demonstrated with a worked example that included a spurious hit. Both programs produced identical and correct results on all 14 test cases.

The measurements confirm the theoretical analysis. On small inputs both algorithms take less than a microsecond and neither is clearly better. On large inputs designed to be difficult for the Naive method, the Naive algorithm needed up to 99.9 million comparisons and 0.85 seconds per call, while Rabin-Karp took less than 2 milliseconds, a speed-up of 103 to 876 times, and its running time grew in proportion to the text length and not to the pattern length. The project also showed that Rabin-Karp is not a universal improvement: it gains only when the saved comparisons outweigh the cost of hashing, and its linear time is an expected value that depends on a good hash function.

9.2 Future Scope

- Use a larger prime modulus (about 10^9) with 64-bit arithmetic, or **double hashing** with two moduli, to make spurious hits extremely unlikely.
- Add the Knuth-Morris-Pratt and Boyer-Moore algorithms to the benchmark to obtain a complete comparison of exact matching methods.
- Benchmark with realistic data such as English text, source code and DNA sequences, and with random text of varying alphabet size, instead of only synthetic worst-case strings.
- Repeat the measurements with compiler optimisation (`-O2`) and report the mean and the standard deviation over several runs.
- Extend Rabin-Karp to multiple patterns and to case-insensitive matching.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009. (Chapter 32: String Matching.)

[2] R. M. Karp and M. O. Rabin, “Efficient randomized pattern-matching algorithms,” *IBM Journal of Research and Development*, vol. 31, no. 2, pp. 249–260, 1987.

[3] D. E. Knuth, J. H. Morris Jr. and V. R. Pratt, “Fast pattern matching in strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.

[4] R. S. Boyer and J. S. Moore, “A fast string searching algorithm,” *Communications of the ACM*, vol. 20, no. 10, pp. 762–772, 1977.

[5] A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed., Pearson, 2012.

[6] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed., Addison-Wesley, 2011.

[7] cppreference.com, "Date and time utilities (std::chrono)," https://en.cppreference.com/w/cpp/chrono